Exploring Unknown Indoors with Drone using Multi-Critic DDPG Architecture

Kaushal Kishore, Chimata Anudeep, Shahid Shaikh, Samarth Singh

***Abstract*—This paper proposes a novel Deep Deterministic Policy Gradient (DDPG)-based algorithm for autonomous indoor drone navigation in confined and unmapped environments. The proposed approach leverages a multilayered DDPG architecture with a unique reward structure to enable the drone to navigate, adapt, and respond to its surroundings effectively. The reward structure caters to three distinct elements of drone navigation control: 1) obstacle avoidance through artificial repulsive potential hills, 2) center alignment via artificial attraction valleys, and 3) variable pitch control for accelerated exploration. With a multicontrol approach, we can give the drone a higher degree of autonomy to explore an unmapped environment effectively. This paper has discussed the detailed mathematical formulation of the DDPG architecture and hybrid reward structure. To test the proposed navigation algorithm, an extensive evaluation is carried out in a simulation environment. The results demonstrate the effectiveness of the proposed approach in achieving safe and efficient navigation in complex indoor environments.**

The impact statement should not exceeed 150 words. This section offers an example that is expanded to have only and just 150 words to demonstrate the point. Here is an example on how to write an appropriate impact statement: Chatbots are a popular technology in online interaction. They reduce the load on human support teams and offer continuous 24-7 support to customers. However, recent usability research has demonstrated that 30% of customers are unhappy with current chatbots due to their poor conversational capabilities and inability to emotionally engage customers. The natural language algorithms we introduce in this paper overcame these limitations. With a significant increase in user satisfaction to 92% after adopting our algorithms, the technology is ready to support users in a wide variety of applications including government front shops, automatic tellers, and the gaming industry. It could offer an alternative way of interaction for some physically disable users.

***Index Terms*—Drone navigation, Reinforcement learning, DDPG, Hybrid reward structure, Indoor environments**

I. INTRODUCTION

The significance of indoor navigation has surged recently, driven by the urbanization phenomenon and the emergence of novel technologies such as LiDAR, stereo cameras, etc.; such enhancements and lifestyle changes are now driving needs such as autonomous delivery services, logistics [1], cave, tunnel, mines and high-structure indoor mapping, and diverse applications [2]. Research in indoor navigation plays a pivotal role in advancing path-planning methodologies and algorithms to guide agents through these intricate spaces optimally, economically, and with time efficiency. Indoor environments present a complex and dynamic landscape for drones, characterized by narrow corridors and obstacles that pose challenges for autonomous drones. Navigating safely and efficiently within these spaces requires a sophisticated fusion of sensor technologies, path-planning strategies, and control systems.

Several strategies have been used in drone navigation in indoor environments to tackle the challenges of precise and agile maneuvering within confined spaces. These strategies include traditional methods such as Guidance Navigation Control (GNC) [3] using way-point, Visio-Interial (VIO) [4], marker-assisted navigation [5], and Reinforcement Learning (RL) techniques. GNC presents a simplistic approach to drone navigation and is often assisted with VIO and pre-built maps for successful navigation. The same applies to marker-based navigation requiring pre-installed markers for any maneuver. With the added complexity of mapped and unmapped obstacles, recent advanced drone navigation methods have shifted towards AI/ML-based techniques.

Deep Deterministic Policy Gradient (DDPG), a reinforcement learning algorithm, harnesses the power of deep neural networks to learn complex control policies from experience [6]. Its ability to adapt to changing conditions, handle high-dimensional state and action spaces, and provide fine-grained control stands out. Moreover, DDPG offers the potential for continuous improvement through algorithm architectural refinement.

We introduce a novel DDPG-based navigation algorithm for indoor navigation. The navigation architecture controls three distinct degrees of movement – pitch, yaw, and roll. This enhancement empowers the drone to execute intricate maneuvers seamlessly within the confined spaces of indoor environments. To achieve such results, we have tailored a unique reward-penalty framework for navigating a drone in indoor spaces. We also added a set of memory buffers called precision playback that ensures fast and efficient training of drones for navigating in indoor areas. Moreover, our research presents an innovative architectural framework for training the DDPG-based agents that will greatly benefit the research community.

II. RELATED WORK

Reinforcement learning has emerged as a compelling paradigm for achieving autonomous navigation capabilities in

Kaushal Kishore and Samarth Singh are with CSIR-Central Electronics Engineering Research, Pilani-333031 and CSIR-Advanced Materials and Processes Research Institute, Bhopal-462026, respectively.

Chimata Anudeep and Shahid Shaikh are with BITS-Pilani, Goa Campus.

drones. Navigation algorithms within this realm can be broadly categorized based on their state and action spaces: Limited states with discrete actions, Unlimited states with discrete actions, and Unlimited states with continuous actions [7] [1]. In this study, we focus on the latter category, aiming to harness the potential of unlimited states and continuous actions for effective drone navigation.

Within the drone navigation domain based on reinforcement learning, various algorithms have been developed to tackle different scenarios. Notable among these are Trust Region Policy Optimization (TRPO), Stein Variational Policy Gradient (SVPG), Proximal Policy Optimization (PPO), Recurrent Deterministic Policy Gradients (RDPG), Soft Actor-Critic (SAC), and DDPG. TRPO is known for its stability and safety in policy optimization [8], which makes it suitable for critical scenarios such as crowded environments. SVPG uses Stein variational gradient descent to update the policy [9]. PPO is widely used for its simplicity and effectiveness [10], optimizing policies by iteratively improving them while maintaining a proximal policy update constraint. RDPG incorporates recurrent neural networks (RNNs) into the policy network [11], capturing temporal dependencies in the drone's actions, making it fit for tasks needing memory of past states, such as complex indoor navigation. SAC optimizes both the policy and value function and uses entropy regularization to encourage exploration in challenging scenarios [12] [7].

Among these, the DDPG algorithm has attracted considerable attention for handling continuous action spaces [1]. This research adopts a dense reward structure that provides continuous feedback to the agent in each state [13]. This approach ensures that the agent can accurately gauge the accuracy of its actions throughout its trajectory, contributing to faster and more effective learning.

III. Proposed Solution

This research paper delves into indoor drone navigation, focusing on DDPG to enable the drone's seamless maneuvering within intricate and uncharted corridor-like environments [6]. The essence of our work lies in developing a localized path planning system, driven by reinforcement learning, that empowers the drone to navigate, adapt, and respond to its surroundings autonomously.

By introducing the novel architecture of multi-critic DDPG framework and the Hybrid Reward Structure, we seek to enhance the capabilities of reinforcement learning algorithms in addressing complex real-world navigation challenges. We propose a three-fold reward structure for training the DDPG-driven agent, addressing obstacle avoidance, center alignment, and variable pitch. Our approach uses artificial potential fields [14] to create repulsive hills for obstacle avoidance and attraction hills for center alignment. Additionally, we incorporate variable pitch by understanding the environment making the drone accelerate forward in obstacle-free areas, enabling rapid exploration. Inspired by well-established robotics principles, this Hybrid Reward Structure aims to help the agent avoid collisions and maintain optimal path alignment.

The proposed solution is divided into logical sections, which have been explained mathematically and validated in simulation as well. The results obtained were followed by a discussion and conclusion.

The foundation of our methodology rests upon the DDPG Algorithm, which is rooted in the Actor-Critic Network framework. This paradigm uses the principle of Temporal Difference (TD) error for training the $Critic$, while simultaneously employing the negative of the Q-value to refine the $Actor$ [6].

Our algorithm starts with an $Actor$ that generates output for three action values: pitch, yaw, and roll. We train this network by employing a cluster of three distinct $Critic$ components. These specialized $Critic$ networks autonomously evaluate three distinct actions, thereby bifurcating the evaluation process into the individual dimensions of movement.

1) $Multi-Action-System$*:* The $Actor$ component plays the role of generating action to be executed. While training, the output of the $Actor$ is subsequently evaluated by the $Critic$ to determine its associated Q-value for a given state-action pair.

The $Actor$ is trained using a loss function defined as the summation of $-Q$ value output by the respective $Critic's$ evaluation described in Eq. (1). This loss is averaged across the sampled experiences, and the gradient of the loss function is calculated using Eq. (2). Using this gradient, the parameters of the $Actor$ are updated using the gradient descent algorithm as shown in Eq. (3) with α as the learning rate.

$$Loss(J_t) = \frac{\sum_{m=1}^{3}\sum_{t=1}^{k}(-Q_{\theta_m^Q}(s_t, a_t))}{3k} \tag{1}$$

Here, k is the number of samples taken in a batch from the buffer to calculate the loss, θ_m^Q denotes the weight of m^{th} $Critic$ network, θ^μ are the parameters of $Actor$ and the subscript t denotes the timestamp of the parameters.

$$\nabla_{\theta_t^\mu} J_t = \frac{-\sum_{m=1}^{3}\sum_{t=1}^{k}\nabla_{a_t} Q_{\theta_m^Q}(s_t, a_t)\nabla_{\theta_t^\mu}\mu(s_t)}{3k} \tag{2}$$

$$\theta_{t+1}^\mu \leftarrow \theta_t^\mu - \alpha\nabla_{\theta_t^\mu} J_t \tag{3}$$

2) $Multi-Critic-System$*:* The algorithm for drone navigation presented here has a dedicated $Multi-Critic$ network for independently evaluating the actions generated by the $Actor$. So, we have three $Critic$ networks each for evaluating the drone's roll, pitch, and yaw motion by using the Bellman Equation and Temporal Difference to calculate the loss in Eq. (4).

$$\delta_t = MSE(R_{(t+1)} + \gamma Q_{\phi_t}(s_{(t+1)}, a_{(t+1)}) - Q_{\theta_t^Q}(s_t, a_t)) \tag{4}$$

$$\theta_{t+1}^Q \leftarrow \theta_t^Q - \alpha_1\nabla_{\theta_t^Q}\delta_t \tag{5}$$

Here, ϕ and θ^Q are the weights of the critic target and critic, γ is the discount factor, δ_t is the loss, α_1 shows the learning rate for the $Critic's$ weights and the subscript t denotes the time stamp of the parameters. This loss is then back-propagated for each $Critic$ separately using their respective reward values as in Eq. 5.

The loss function uses a semi-gradient descent approach to update the weights of the $Muti-Critic$ system using three

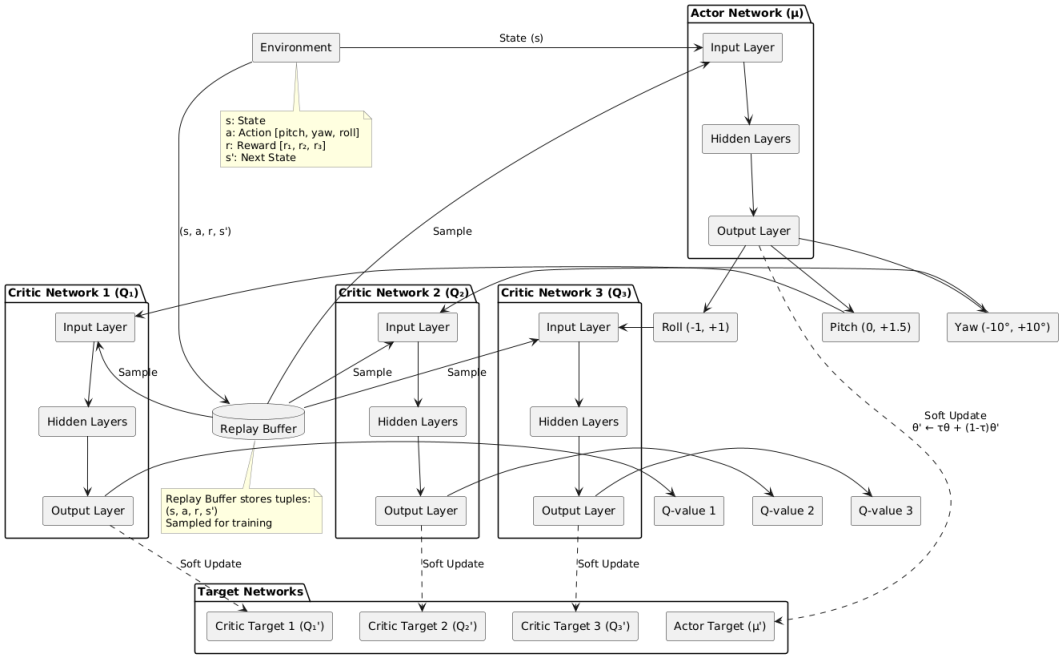

Fig. 1. Multi-layer DDPG architecture for drone control and navigation

$Critic\ Target's$. The $Actor\ Target$ generates the actions for the next state, which are evaluated by the $Critic\ Target$ networks to find the Q values for those actions.

We have adopted a soft update approach for the $Critic\ Target$ weights (ϕ) to ensure an adaptable and efficient learning process. This is done by updating the weights with a degree of smoothness (τ) as shown in Eq. (6), instead of directly copying the weights of $Critic's$, thus helping to maintain stability during the learning process.

$$\phi_{t+1} \leftarrow \tau\theta^{Q}_{t+1} + (1-\tau)\phi_t \tag{6}$$

3) Hybrid Reward Structure: The first among these rewards is $reward_1$, which is used to evaluate the value of $pitch$ by $Actor$. We have introduced a variable $pitch$ for enhanced agility. Within this construct, a threshold of 1.5 m/s is set as the upper limit for the attainable pitch values. This constraint effectively confines the pitch adjustments of the drone within an interval of [0, 1.5] m/s.

$$reward_1 = \begin{cases} \beta(F - F_t)^2 & ; \quad F_t < F \\ -\eta * e^{\frac{1}{(F-F_t)+\epsilon}} & ; \quad F_t \geq f \end{cases} \tag{7}$$

Here, F_t stands for threshold value in the forward range with respect to the obstacle, F is the real-time forward range, ϵ is an infinitely small value to avoid undefined state, β and η are real variables which are hyper-parameters greater than zero.

The second reward is named $reward_2$, designed for the drone to avoid obstacles. This integral construct operates in tandem with the notion of artificial potential hills [14].

$reward_2$ operates by transforming the sensed obstacles into virtual repulsive potential hills.

The magnitude of repulsion, which is the value assigned to the drone's state by the reward, is inversely proportional to the distance of the drone from the walls/obstacle. The mathematical repulsion field can be described in Eq. (8).

$$U_{rep} = \begin{cases} \lambda\left(\frac{1}{d} - \frac{1}{d_o}\right)^2 + 1 & ; \quad d \leq d_o \\ 1 & ; \quad d > d_o \end{cases} \tag{8}$$

$$rep\_eq_t = \begin{cases} \frac{U_{rep,R}}{U_{rep,L}} & ; U_{rep,R} > U_{rep,L} \\ \frac{U_{rep,L}}{U_{rep,R}} & ; U_{rep,R} \leq U_{rep,R} \end{cases} \tag{9}$$

Here U_{rep} is the repulsive potential received by the agent when the distance d is less than d_o, λ is the parameter set while training the agent. The repulsion equilibrium function is defined to quantify the imbalance between the repulsion received from both sides of the drone. The drone uses a LiDAR range of $[-90^o, 90^o]$. Out of this range, $[-10^o, 10^o]$ is used as the forward section. Values beyond 10^o and -10^o are used as right and left ranges, respectively, as shown in Fig. 2. The minimum values for the right section are indicated by r, and the minimum value from the left section is indicated by l, which means the immediate obstacles in these respective areas. $Reward_2$ is based on the change in repulsion equilibrium ($rep_{Eq.}$) due to the action output by the $Actor$ as shown in Eq. (11).

$$\Delta rep_{Eq.} = rep_{eq_{t+1}} - rep_{eq_t} \tag{10}$$

According to the function definition of rep_{eq_t}, the repulsion equilibrium function is always > 1. So if $\Delta rep_{Eq.} < 0$ it would show that the agent is heading in the right direction

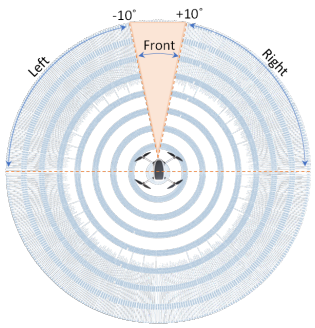

Fig. 2. Drone LiDAR scan spatial division for navigation. The front, left, and right sectors around the drone are marked in the image.

and thus would be awarded as per Eq. (11).

$$reward_2 = \begin{cases} \frac{4\kappa}{rep_{eq_{t+1}}}, & \text{if } rep_{eq_{t+1}} < \kappa \text{ and } \Delta rep_{eq.} < 0 \\ \frac{\kappa}{rep_{eq_{t+1}}}, & \text{if } rep_{eq_{t+1}} < \kappa \text{ and } \Delta rep_{eq.} > 0 \\ -\omega e^{\frac{-1}{rep_{eq_{t+1}} - \rho}}, & \text{if } rep_{eq_{t+1}} > \kappa \end{cases} \tag{11}$$

Here κ, ω, and ρ are hyperparameters chosen while training.

$Reward_3$ aligns the drone's trajectory with the central axis of the corridor by working in tandem with $reward_2$ to attain a faster repulsion equilibrium. This is done by minimising the difference between R and L as shown in Eq. (12). To accomplish this, we have introduced the concept of artificial attraction valley in this work. Just as the drone perceives obstacles as repulsive potential hills in $reward_2$, it now perceives the central axis as an attractive potential valley in $reward_3$.

$$U_{attr} = -\mu(R - L)^2 + \delta \tag{12}$$

$$\Delta_{attr} = U_{attr,t+1} - U_{attr,t} \tag{13}$$

U_{attr} is the amount of attraction the drone receives from the centre to have a centre alignment, and (R, L) is the distance from the nearest obstacle in the right and left range, respectively. μ and δ are hyper-parameters greater than zero. The parameter Δ_{attr} as in Eq. (13) serves as a discerning metric, allowing us to gauge the accuracy and correctness of decisions made by the $Actor$ with regard to central axis adjustments. The reward structure presented in Eq. (14) ensures that deviation from the central alignment yields diminishing rewards, thereby incentivizing the drone to execute precise centred movements.

$$reward_3 = \begin{cases} 2U_{attr,t+1} & ; \Delta_{attr} > 0 \\ U_{attr,t} & ; \Delta_{attr} < 0 \end{cases} \tag{14}$$

4) Precision Playback: The Memory Buffer is a crucial training tool for the $Actor$, with updates occurring at regular intervals of 10-time steps. It accumulates experiences when the loss associated with the current state s_t is lower than that of the preceding state s_{t-1}. Precision Playback PP emerges as a specialised memory buffer variant designed to house and prioritise the most optimal experiences within this framework. This includes states with a repulsion equilibrium smaller than κ as defined in $reward_2$ eq (11). The essence of the PP Buffer is rooted in training the $Actor$ with prime-quality samples, thereby expediting the neural network's learning trajectory toward the desired objectives.

By effectively combining these distinct reward structures and PP buffers, our methodology fosters a holistic navigation strategy that encapsulates dynamic pitch adjustments, astute obstacle avoidance, and precise centre alignment.

IV. Algorithm Validation in Simulation

The discussed DDPG architecture for drone navigation control is implemented in ROS2 [15] for evaluation. We used the Iris drone with PX4-SITL [16] inside a virtual world designed in Gazebo-11. The drone within the simulated environment is equipped with a 360^o laser range finder. We modified the SITL gazebo sensor model to match our algorithm inputs by saturating the ranges beyond 6m. We designed a closed passage with varying widths and turns for the training world (shown in Fig. 3). We also added wall extensions and pillars irregularly to make the training world more complex. This makes the learning phase efficient by generating complex control commands from the DDPG architecture.

A. Training Algorithm

As outlined in Algorithm 1, the training algorithm begins with initialising three $Critic$ networks, three $Critic\ Targets$, one $Actor$, and an $Actor\ Target$ (Algorithm 1, line 2). Each episode unfolds over a predetermined number of steps (Algorithm 1, lines 4-5). The journey begins with the capture of the agent's initial state through the LiDAR sensor, translating into a state vector $[F, R, L]$, denoting the nearest obstacle distances in the forward, right, and left ranges, respectively (Algorithm 1, line 7).

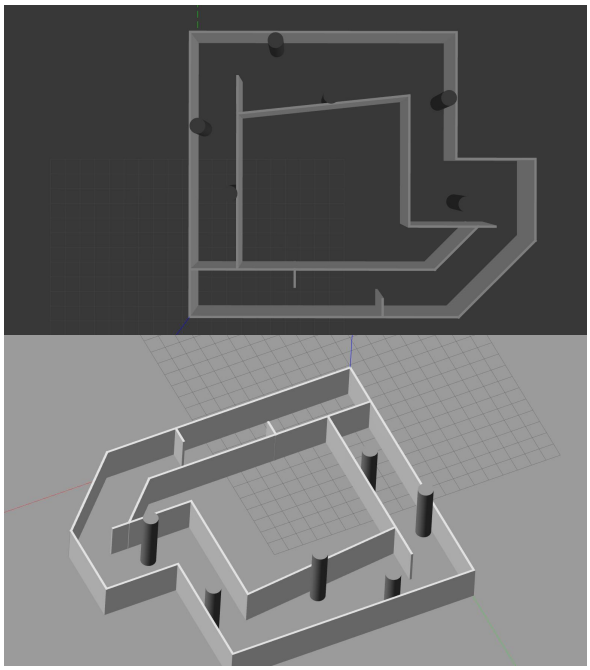

Fig. 3. Drone training arena in gazebo simulation world

Algorithm 1: Proposed DDPG based training

1 Init $CRITIC_1$; $CRITIC_2$; $CRITIC_3$
2 Init $ACTOR$
3 Drone take off;
4 **for** $1 : N\_ep$ **do**
5 **for** $1 : N\_step$ **do**
6 **if** $land == 0$ **then**
7 $s_t = [F, R, L]$;
8 $pitch_t, yaw_t, roll_t = ACTOR(s) + N$;
9 Observe, s_{t+1};
10 Observe rewards $r1, r2, r3$ (using Eq.7 to Eq. 10) ;
11 $q1_t$ = $CRITIC1(s_t, pitch_t)$;
12 $q2_t$ = $CRITIC2(s_t, yaw_t)$;
13 $q3_t$ = $CRITIC3(s_t, roll_t)$;
14 $pitch_{t+1}, yaw_{t+1}, roll_{t+1} = ACTOR\ TGT(s_{t+1})$;
15 $q1_{t+1}$ = $CRITIC\_TARGET_1(s_{t+1}, pitch_{t+1})$;
16 $q2_{t+1}$ = $CRITIC\_TARGET_2(s_{t+1}, yaw_{t+1})$;
17 $q3_{t+1}$ = $CRITIC\_TARGET_3(s_{t+1}, roll_{t+1})$;
18 R_t = $[r1_t, r2_t, r3_t]$, A_t = $[pitch_t, yaw_t, roll_t]$;
19 Q_t = $[q1_t, q2_t, q3_t]$; (from $eq.$ 4 get $\delta_1, \delta_2, \delta_3$);
20 $\delta_t = \delta 1_t + \delta 2_t + \delta 3_t$
21 Train $CRITIC's$ with Eq. 4 using $\nabla \delta_t$;
22 $\delta_C = \delta_t - \delta_{t-1}$
23 **if** $\delta_C < 0$ **then**
24 $m = [s_t, A_t, s_{t+1}, R_t, \delta_t]$;
25 **if** $bal_{t+1} <= \kappa$ **then**
26 **if** $\left.\begin{matrix}|F - F^*| \\ |L - L^*| \\ |R - R^*|\end{matrix}\right\} < 0.2$ *and* $\delta_t < \delta_{t^*}$;
27 **then**
28 Replace memory in PP with m
29 **else if** m *is new;*
30 **then**
31 $PP \leftarrow$m
32 **else**
33 No memory saved
34 **end**
35 **end**
36 RE $\leftarrow m$ **end**
37 **else**
38 Drone land;
39 **end**
40 **if** n_step *%5 == 0* **then**
41 Soft update weights of critic targets using Eq.5;
42 Save weights of Critics, Critic_Targets and Actor;
43 **if** n_step *%10 == 0* **then**
44 **if** *len(PP)* $\geq$ *12* **then**
45 Tr_b PP;
46 Tr_b $random(RE(60 - len(PP)))$;
47 **else**
48 $random(RE(60))$
49 **end**
50 Train ACTOR with batch:Tr_b using $eq.$ 4;
51 Call Test; Calculate average rewards per episode;
52 **end**
53 Empty RE;
54 **end**
55

Subsequently, this state vector is presented to the $Actor$ network, generating the pivotal action trio of $pitch_t$, yaw_t and $roll_t$ (Algorithm 1, line 8). To infuse a spirit of exploration, a Gaussian noise component is injected into these actions (Algorithm 1, line 8), subsequently guiding the drone's execution. The executed action propels the agent to a new state s_{t+1} (Algorithm 1, line 9) s_{t+1}, is used for the computation of three distinct rewards via Eq.'s (7), (11), (13) (Algorithm 1, line 10).

Critical to the training process, the Q_t values are evaluated for all actions undertaken by the $Critic$ networks, labelled as $q1_t$, $q2_t$, and $q3_t$ (Algorithm 1, lines 11-13). The cycle progresses, allowing the algorithm to calculate the actions $pitch_{t+1}$, yaw_{t+1}, and $roll_{t+1}$ using s_{t+1} as input to the $Actor\ Target$ network (Algorithm 1, line 14). Consequently, the $Critic\ Target$ networks step in, calculating the Q_{t+1} values for the respective actions and denoting them as $q1_{t+1}$, $q2_{t+1}$, and $q3_{t+1}$ (Algorithm 1, lines 15-17).

The losses for all $Critic$ networks are computed through Eq. (4), culminating in deriving a combined loss, aggregating individual critic losses (Algorithm 1, lines 21-22). This collective loss drives the backpropagation process, which subsequently updates the parameters of the $Critic$ networks (Algorithm 1, line 23). The algorithm determines the storage of experience samples within the memory buffer and the precision playback (Algorithm 1, lines 25-40). A key facet involves the soft updating of the $Critic - Target$ weights every 5-time steps, governed by Eq. (6), followed by weight preservation (Algorithm 1, lines 43-45).

At every 10-time step, a randomised selection of 60 experiences is drawn from the memory buffer and PP buffer to train the $Actor$ (Algorithm 1, lines 46-53). The $Actor's$ loss is computed through Equation 1, independently for each of the three $Critic's$. Subsequently, the aggregated loss from these calculations is back-propagated across the $Actor$ network, effecting weight updates that refine the $Actor's$ decision-making prowess. Notably, a minimum of 12 samples is selectively picked from the precision playback. Concluding the iterative process, the test algorithm is invoked, thereby evaluating the trained network's performance and calculating the average reward per episode (Algorithm 1, line 54).

B. Testing Algorithm

During the evaluation phase, the $Actor's$ weights are initialised for testing. Following takeoff, the drone determines the state, relying on input data from the LiDAR sensor (Algorithm 2, line 4). Subsequently, it assesses its proximity to obstacles, with the prescribed safety range set at 1 m in our proposed methodology (Algorithm 2, line 5). When the drone detects obstacles within a range of less than 1 m, it dynamically adjusts its direction, favoring the side-either right (r) or left (l)-with the greater distance from the obstacles. Conversely, if it exceeds 1 m, the input data is transmitted to the $Actor$ to generate three crucial action commands: $pitch$, yaw, and $roll$ (Algorithm 2, lines 13-16).

Furthermore, the algorithm discerns deployment and testing modes (Algorithm 2, line 18). In the testing mode, it calculates the reward achieved during execution, thus enabling the computation of the average reward per episode. This delineation ensures robust evaluation and performance assessment.

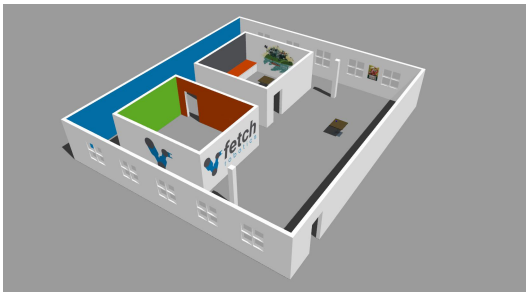

Fig. 4. Gazebo world for testing the trained agent.

Algorithm 2: Trained DDPG network: test and deployment

```
Load trained actor network:ACTOR
Drone take off;
if land==0 then
    Input = [F, R, L];
    if F < F_o then ;   // F_o is front sector threshold

        Let x, x' = L, R or R, L if x > x' then
            Turn_dir = x;
        else
            Turn_dir = x';
        end
    else
        action = ACTOR(Input);
        surge pitch = action[0]; ∈ [0m, 1.5m]
        surge yaw = action[1]; ∈ [−10°, 10°]
        surge roll = action[2]; ∈ [−1m, 1m]
    end
    if Test==True then ;        // testing during training

        observe r for computing average rewards;
    else ;                      // Deploying trained actor

        Under Deployment;
    end
else
    Drone land;
end
```

C. Policy to Action Mapping

The *Actor*'s outputs consist of pitch, yaw, and roll. Each dimension has a defined range: for pitch, it spans from [0, 1.5] m/s; for yaw, from [-10°,10°]; and for roll, from [-1, 1] m/s.

At the beginning of its flight, the drone uses its Lidar sensor to measure the distance to the closest obstacles in front, to the right, and to the left. This information is then sent to the Actor-network, which calculates and outputs the exact values needed for the drone's pitch, yaw and roll. The Lidar sensor's range is limited to 6 m to avoid errors caused by extremely faraway readings.

V. Results

A. DDPG Training Convergence and drone navigation

The training environment consisted of a corridor-like structure with interspersed cylinders to simulate obstacle avoidance scenarios. The drone followed the control signals generated by the actor-network, encompassing roll, pitch, and yaw adjustments. Using the hybrid reward structure led to smoother and more stable trajectories during navigation. We observed reduced trajectory deviations and oscillations, indicating enhanced control and precision in the drone's movements as shown in Fig. 5(a-c). We can see the different stages the drone has gone through during the training phase. We spawned the drone from different locations during the training phase to eliminate the bias induced by a similar starting point. Our training process analysis indicated that the actor and critic networks converged during training, as shown in Fig. 5(e). The mean of the three critic outputs consistently decreased over training iterations, reflecting the effectiveness of the training process.

B. Performance in testing environment

The testing environment comprised two closed rooms, each secured with a door. The drone successfully navigated the environment while avoiding obstacles and reaching the endpoint. This achievement was enabled by its ability to efficiently utilise the full range of its motion (including roll, pitch, and yaw) and exploit the extended pitch range to cover longer distances in obstacle-free areas.

Subsequently, upon encountering an open doorway (indicating the absence of obstacles in that direction), the drone successfully exited the room. After aligning itself once again, it navigated out of the entire environment.

C. Comparison and Discussion

This work on Deep Deterministic Policy Gradient (DDPG)-based drone navigation system has been an enhancement over the previously developed Continuous Actor-Critic Learning Automaton (CACLA) approach [13] from our group. We addressed critical factors in drone navigation and presented a redesigned navigation architecture working in all three degrees of freedom, with a hybrid reward structure that could offer advantages such as:

1) Enhanced Agility: DDPG-based drones exhibited superior agility due to their ability to perform roll and pitch adjustments with greater degrees of freedom compared to the limited maneuverability of CACLA drones. This enabled them to adapt more effectively to diverse environmental conditions.

2) Optimized Decision-Making: The Hybrid Reward Structure guides the drone toward optimal decisions through a combination of obstacle avoidance. The variable pitch capability of DDPG drones empowered them to make more optimal decisions, particularly in obstacle-free scenarios. By adjusting their pitch angle for increased forward velocity, they could efficiently cover longer distances, resulting in faster navigation.

3) Adaptability to Uncertainties: The DDPG algorithm's inherent reinforcement learning nature allows the drone to dynamically adjust its behaviour based on encountered obstacles and environmental changes, fostering robustness in real-world scenarios.

Our study on the DDPG-based drone navigation system demonstrates several key advancements over existing methods in the field, as highlighted in Table I.

Compared to the CACLA algorithm with special experience replay [13], which dynamically adapts to new environments but is limited to pitch and yaw movements, our DDPG-based system includes roll adjustments. This enhancement allows for superior maneuverability and reduced navigation time.

When contrasted with the POMDP and MDP hybrid approach [17], which enhances obstacle avoidance and goal achievement by combining local and global planning models, our method's unified DDPG approach simplifies integration and improves adaptability. This streamlined framework enables dynamic adjustments to environmental changes, enhancing robustness without the complexity of managing separate models. Although DQN [18] can generate 3D paths and navigate obstacles, it is restricted by the discrete action space. In contrast, our DDPG algorithm handles continuous action spaces, providing more fluid and precise navigation capabilities.

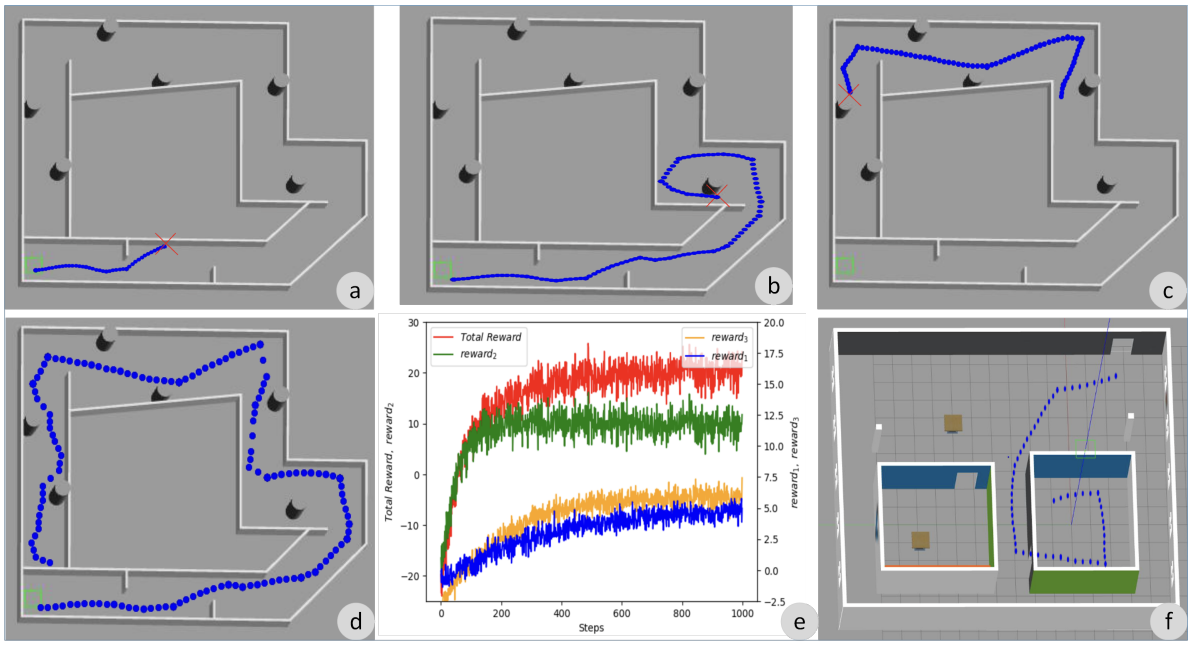

Fig. 5. a),b),c) Show plots of the drones while training. d) Plot of the path taken by drone after there was a convergence in the reward functions in the training environment. f) Plot of the path taken by drone in the test environment. e) Plot of the average rewards per episode as the number of episodes increases

TABLE I
HIGHLIGHTS OF RELATED WORK IN THE DRONE NAVIGATION AREA

Reference	Research objective	Approach	Strength	Weakness
[13]	Autonomous indoor drone navigation	CACLA Algorithm, Special Experience Replay	Dynamically adapt to new environments	Increasing navigation time by having pitch and yaw as the only possible movements
[17]	UAV Navigation in Indoor Environment	Partially Observable Markov Decision Processes (POMDP) for local planning and Markov Decision Process (MDP) for global planning	Combining MDP for global planning and POMDP for local environments enhances obstacle avoidance and goal achievement for the drone	Relies on separate MDP and POMDP models, potentially complicating integration and adaptability compared to unified approaches
[18]	Drone Navigation in 3D Landscape	RGB Image, Depth Map, CNN and DQN Network	Able to generate 3D paths with Depth Map and navigate 3D obstacles	Uses DQN which is mainly used for discrete action space
[19]	Obstacle Avoidance of drones	Actor Critic Algorithm with Unet segmentation of images	Real-time label map generation via optical flow replaces manual labeling	Struggles in low-light or uniform-colored environments, hindering accurate segmentation and navigation
[20]	Drone Navigation using sensor data	Proximal Policy Optimisation with Long short-term memory (LSTM) Layers	Employs LSTM neural networks to recall past states, enabling optimal action when states recur	LSTM Layers increase complexity and training time
[21]	Trail Navigation using MAV's	Novel TrailNet Architecture, vision-based approach	Efficiently navigate unstructured environments	Computationally expensive to process lot of image data, Not able to navigate at night
This work	Navigating indoors autonomously with drones	DDPG algorithm, Hybrid Reward Structure 3 degrees of motion	Can dynamically adapt to environment and can roll, pitch and yaw saving time. Ideal for companion computer level computation	Increased training time

Compared to the actor-critic algorithm with Unet segmentation [19], which excels in the generation of real-time label maps but struggles in low-light or uniform-colored environments, our method ensures consistent performance in varied lighting conditions with a robust hybrid reward structure. The use of Proximal Policy Optimization (PPO) with LSTM layers [20] allows for the recall of past states, optimizing actions based on historical data. However, this increases the complexity and training time, which is balanced by our DDPG algorithm. Lastly, compared to the vision-based TrailNet architecture for trail navigation using MAVs [21], which efficiently handles unstructured environments but is computationally expensive and ineffective at night, our approach is computationally efficient for companion-level computing and maintains high performance in various indoor settings. In general, our DDPG-based navigation system with its hybrid reward structure presents significant improvements in agility, decision-making, and adaptability.

VI. CONCLUSION

This research explored the domain of indoor drone navigation, taking advantage of the prowess of DDPG to endow drones with the ability to maneuver seamlessly within intricate and uncharted environments. We introduced a novel DDPG-based navigation algorithm characterized by three distinct degrees of movement (pitch, yaw, and roll) and a meticulously crafted Hybrid Reward Structure. This structure incorporates artificial potential fields, enabling the drone to effectively avoid obstacles, align itself with the center axis of corridors, and dynamically adapt to its surroundings. Our experiments demonstrated significantly improved navigation success rates compared to conventional DDPG approaches. Including a hybrid reward structure played a crucial role in enhancing the agent's ability to adapt to complex environments. This

structure facilitated effective training by guiding the agent to accurately evaluate and respond to value signals from all three spatial directions (forward, left, and right). This, in turn, contributed significantly to the overall success of the navigation task.

ACKNOWLEDGMENT

The authors thank the Director, CSIR-CEERI, for providing the necessary research infrastructure. This research work is supported by the i-Hub Foundation for Cobotics (IHFC), New Delhi.

REFERENCES

[1] O. Bouhamed, H. Ghazzai, H. Besbes, and Y. Massoud, "Autonomous uav navigation: A ddpg-based deep reinforcement learning approach," 3 2020. [Online]. Available: http://arxiv.org/abs/2003.10923

[2] B. Zhu, E. Bedeer, H. H. Nguyen, R. Barton, and J. Henry, "Uav trajectory planning in wireless sensor networks for energy consumption minimization by deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, vol. 70, pp. 9540–9554, 9 2021.

[3] Y. Li, M. Scanavino, E. Capello, F. Dabbene, G. Guglieri, and A. Vilardi, "A novel distributed architecture for UAV indoor navigation," *Transportation Research Procedia*, vol. 35, pp. 13–22, 2018.

[4] J. Tiemann, F. Schweikowski, and C. Wietfeld, "Design of an UWB indoor-positioning system for UAV navigation in GNSS-denied environments," in *2015 International Conference on Indoor Positioning and Indoor Navigation (IPIN)*. IEEE, oct 2015.

[5] M. Irfan, S. Dalai, K. Kishore, S. Singh, and S. Akbar, "Vision-based guidance and navigation for autonomous MAV in indoor environment," in *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*. IEEE, jul 2020.

[6] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 9 2015. [Online]. Available: http://arxiv.org/abs/1509.02971

[7] F. AlMahamid and K. Grolinger, "Autonomous unmanned aerial vehicle navigation using reinforcement learning: A systematic review," 10 2022.

[8] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust region policy optimization," 2017.

[9] Q. Liu and D. Wang, "Stein variational gradient descent: A general purpose bayesian inference algorithm," 2019.

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[11] D. Wierstra, A. Förster, J. Peters, and J. Schmidhuber, "Recurrent policy gradients," *Logic Journal of the IGPL*, vol. 18, pp. 620–634, 10 2010.

[12] M. H. Lee and J. Moon, "Deep reinforcement learning-based uav navigation and control: A soft actor-critic with hindsight experience replay approach," 2021.

[13] S. Singh, K. Kishore, S. Dalai, M. Irfan, S. Singh, S. A. Akbar, G. Sachdeva, and R. Yechangunja, "Cacla-based local path planner for drones navigating unknown indoor corridors," *IEEE Intelligent Systems*, vol. 37, pp. 32–41, 2022.

[14] M. G. Y. D. Park, J. H. Jeon, and M. C. Lee, "Obstacle avoidance for mobile robots using artificial potential field approach with simulated annealing."

[15] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022. [Online]. Available: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074

[16] PX4, "Px4/firmware," Feb 2020. [Online]. Available: https://github.com/PX4/Firmware

[17] O. Walker, F. Vanegas, F. Gonzalez, and S. Koenig, "A deep reinforcement learning framework for uav navigation in indoor environments," in *2019 IEEE Aerospace Conference*. IEEE, Mar. 2019.

[18] S.-Y. Shin, Y.-W. Kang, and Y.-G. Kim, "Automatic drone navigation in realistic 3d landscapes using deep reinforcement learning," in *2019 6th International Conference on Control, Decision and Information Technologies (CoDIT)*, 2019, pp. 1072–1077.

[19] S. Y. Shin, Y. W. Kang, and Y. G. Kim, "Reward-driven u-net training for obstacle avoidance drone," *Expert Systems with Applications*, vol. 143, 4 2020.

[20] V. J. Hodge, R. Hawkins, and R. Alexander, "Deep reinforcement learning for drone navigation using sensor data," *Neural Computing and Applications*, vol. 33, pp. 2015–2033, 3 2021.

[21] N. Smolyanskiy, A. Kamenev, J. Smith, and S. Birchfield, "Toward low-flying autonomous mav trail navigation using deep neural networks for environmental awareness," 5 2017. [Online]. Available: http://arxiv.org/abs/1705.02550